

Intro to HTML, CSS, and JS

A Beginner-Friendly Guide to Reading Web Code

Document Syllabus

This document is organized into two main sections. We begin with a simple introduction to HTML, CSS, and JavaScript, then move into the basic syntax we need to read and understand simple web code.

1. Introduction

- What HTML, CSS, and JavaScript are
- How the three languages work together
- The body analogy: structure, appearance, and behavior
- A simple visual example

2. Understanding the Basic Syntax

• HTML Syntax: Creating the Parts

- Definition of HTML syntax
- Common HTML tags
- A simple HTML activity example
- Default HTML layout
- Block-level and inline-level elements
- The `<br>` tag
- Tag attributes: adding extra details

• CSS Syntax: Styling the Parts

- Definition of CSS syntax
- CSS selectors, properties, and values
- Common CSS properties
- How HTML class names connect to CSS selectors
- Comments in HTML, CSS, and JavaScript
- Adding CSS to a mini activity
- Two ways to add CSS: inline CSS and style block CSS

Intro to HTML, CSS, and JavaScript

In today's digital world, creating interactive and visually appealing content for the web requires knowledge of three core technologies: HTML, CSS, and JavaScript. These three building blocks work together to form the structure, style, and interactivity of web pages and applications.



HTML

HyperText Markup Language

HTML creates the **structure** of web pages and interactive activities. It organizes the main parts of the page, such as text, images, buttons, questions, answer choices, links, and sections.

In simple terms, HTML is the **skeleton** of the page.



CSS

Cascading Style Sheets

CSS controls the **appearance** of a page or activity. It changes visual details such as colors, fonts, spacing, borders, layout, and animations.

In simple terms, CSS is the **clothing** of the page.



JavaScript

Programming Language

JavaScript adds **behavior and functionality** to web pages and activities. It makes the page respond when we click, type, select an answer, drag an item, or complete an action.

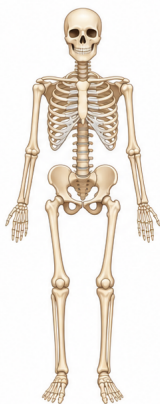
In simple terms, JavaScript is the **brain and nerves** of the page.

To visualize the concepts of HTML, CSS, and JavaScript, we can use the analogy of a human body.

HTML



Structure (Skeleton)



CSS



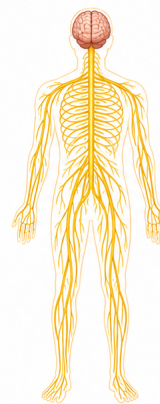
Appearance
(Skin, Eyes, Exterior)



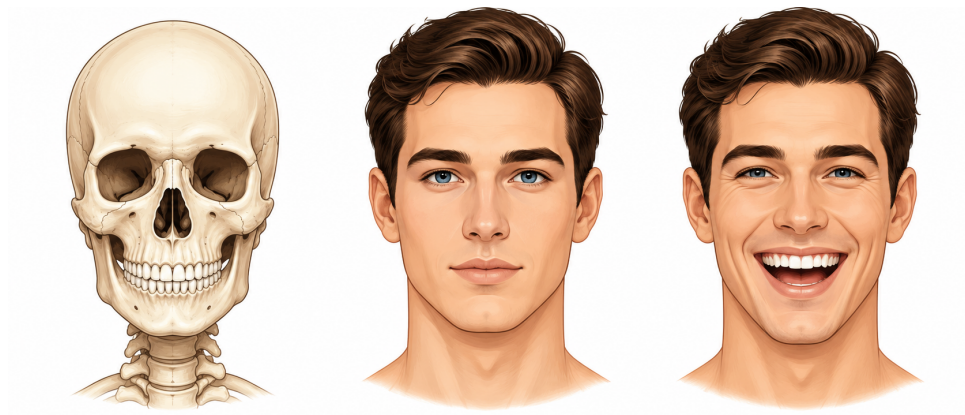
JavaScript



Behavior
(Brain, Nerves, Functionality)



Let's take a closer look with a simple example.



The close-up image shows the same idea in a smaller example.

HTML: The Structure

HTML creates the parts of the activity. In this example, it gives us the face, the eyes, and the mouth. These parts exist before we style them or make them move.

```
<div class="face">
  <div class="hair"></div>
  <div class="eye"></div>
  <div class="nose"></div>
  <div class="mouth"></div>
</div>
```

CSS: The Appearance

CSS changes how the parts look. It can give the face skin color, add hair color, make the eye blue, and shape the mouth.

```
.face {
  background-color: #ffd1b7;
  border-radius: 50%;
}

.hair {
  background-color: brown;
}

.eye {
  background-color: blue;
}

.mouth {
  border-radius: 50%;
}
```

JavaScript: The Behavior

In this example, a JavaScript function can change the mouth when we click the face or press a button, making it look like the face is laughing.

```
function laugh() {
  document
    .querySelector(".mouth")
    .classList.add("laugh");
}
```

Understanding the Basic Syntax

Before we start building small activities, it helps to understand the basic pattern behind the code. We do not need to memorize everything. Instead, we want to recognize what each part is doing so we can read AI-generated code, adjust it when needed, and explain what we want more clearly.

HTML Syntax: Creating the Parts

We can think of HTML as the part of the code that answers the question: **What do we want to place on the screen?**

For example, if we want a paragraph, a button, an image, a table, a choice, or a feedback box, HTML is what creates those pieces. CSS can style them later, and JavaScript can make them respond, but HTML gives us the actual elements to work with.

Most HTML elements are written using **tags**. A tag tells the browser what type of element we are creating.

```
<p>This is a paragraph.</p>
```

In this example, `<p>` opens the paragraph, and `</p>` closes it. The text between the opening and closing tags is **the content that appears on the page**.

```
<tag>Content that appears on the page</tag>
```

Common HTML Tags for Small Activities

- `<div>` creates a general block or container. We often use it to group parts of an activity.
- `<h1>`, `<h2>`, `<h3>` create headings with different sizes.
- `<p>` creates a paragraph of text.
- `<button>` creates a clickable button.
- `<input>` creates a field or choice we can interact with, such as a text box, checkbox, or radio button.
- `<label>` adds readable text beside an input. It helps us understand what each choice means.
- `<img>` adds an image.
- `<table>` creates a table for rows and columns.
- `<tr>` creates a table row.
- `<td>` creates a table cell.
- `<span>` targets a small piece of text inside a line.
- `<br>` creates a line break.

Here is a small example that uses a few common HTML tags together. This example only uses HTML, so it creates the content, but it does not style it yet.

HTML Code

```
<div class="activity-card">
  <h2 class="activity-title">Quick Check</h2>
  <p class="activity-question">Which language creates the
    structure?</p>

  <input class="answer-input" type="radio" id="html" name=
    "language" value="html">
  <label class="answer-label" for="html">HTML</label><br>

  <input class="answer-input" type="radio" id="css" name=
    "language" value="css">
  <label class="answer-label" for="css">CSS</label><br>

  <input class="answer-input" type="radio" id="js" name=
    "language" value="js">
  <label class="answer-label" for="js">JavaScript</label><br>

  <button class="custom-button">Submit</button>
</div>
```

Browser Output



Click the image to open the live HTML page.

When the browser reads plain HTML, it displays the elements in their default layout.

- The browser reads the code in the order it is written, from **top to bottom**.
- Some elements, such as `<div>`, `<h2>`, and `<p>`, are **block-level elements**.

- Block-level elements usually start on a **new line**, so they appear under each other.
- Other elements, such as `<input>`, `<label>`, and `<button>`, are **inline-level elements**.
- Inline-level elements usually stay on the same line and render from **left to right**, as long as there is enough space.
- However, the `<br>` tag creates a line break, so it forces the next element to move to the next line.

In our example, each answer choice uses an `<input>` and a `<label>`. These two appear beside each other because they are inline-level elements. But after each label, we added `<br>`, so the next answer choice starts on a new line.

In this example:

1. The `<div>` groups the activity into one block.
2. The `<h2>` creates the heading.
3. The `<p>` creates the question text.
4. The `<input>` creates each answer choice as a radio button.
5. The `<label>` adds the visible answer text beside each radio button.
6. The `<br>` moves each answer choice to a new line.
7. The `<button>` creates something we can click.

Attention Please!

The `<br>` tag is different from tags like `<p>` or `<div>` because it does not hold content. It only creates a line break. That is why we write `<br>` by itself, without a closing tag.

This simple activity appears at the top-left of the browser without any special spacing, color, or positioning. This is where CSS becomes useful. HTML creates the parts, but CSS helps us move them, style them, space them, and make the activity look more polished.

Tag Attributes: Adding Extra Details

Some HTML tags can include extra information called **attributes**. Attributes help describe how an element should behave, how it should be identified, or how it should connect to other parts of the activity.

We write attributes inside the opening tag.

```
<input class="answer-input" type="radio" id="html" name="language" value="html">
```

In this example, the `<input>` tag creates a choice. The attributes give that choice more meaning:

- `class="answer-input"` gives the input a class name we can style with CSS.
- `type="radio"` tells the browser this input is a radio button.
- `id="html"` gives this specific choice a unique identifier.
- `name="language"` groups the answer choices together.
- `value="html"` stores the value of the selected answer.

A simple way to remember this is: the **tag** creates the element, and the **attributes** give it extra details.

Attention Please!

Different HTML tags can have different attributes, and some attributes are valid for certain tags but not for others. For example, the `type` attribute is valid for `<input>` but not for `<p>`.

CSS Syntax: Styling the Parts

After HTML creates the parts of the activity, CSS helps us decide how those parts should look. We can think of CSS as the part of the code that answers the question: **How should this appear on the screen?**

CSS can control colors, spacing, borders, fonts, size, alignment, and layout. This is why plain HTML can look very simple at first, but the same HTML can look polished once CSS is added.

CSS usually follows this pattern:

```
selector {  
  property: value;  
}
```

1. The **selector** chooses the HTML element we want to style.
2. The **property** tells the browser what visual feature we want to change.
3. The **value** tells the browser how we want that feature to look.

A Small CSS Example

```
.activity-card {  
  background-color: #ffefe9;  
  border: 2px solid #ff4500;  
  border-radius: 12px;  
  padding: 16px;  
  width: 300px;  
}
```

In this example, `.activity-card` selects the HTML element with the class name `activity-card`. The lines inside the curly braces style that activity card.

- `background-color` changes the background color.
- `border` adds a border around the card.
- `border-radius` rounds the corners.
- `padding` adds space inside the card.
- `width` controls how wide the card is.

Attention Please!

The reason CSS knows which part to style is because the HTML has an attribute called `class`.

```
<div class="activity-card">
```

Then CSS uses that class name with a dot:

```
.activity-card {  
  padding: 16px;  
}
```

The dot before `.activity-card` means we are selecting a class. So, `class="activity-card"` in HTML connects to `.activity-card` in CSS.

Other selectors can target elements by their tag name, id, or other attributes. For now, we are using class names because they are a common and simple way to style specific parts of an activity.

Here is the basic idea:

```
.activity-card /* targets class="activity-card" */  
#main-title /* targets id="main-title" */  
button /* targets every <button> tag */
```

Common CSS Properties We May Use

- `color` changes the text color.
- `background-color` changes the background color.
- `font-size` changes the size of the text.
- `font-family` changes the style of the font.
- `padding` adds space inside an element.
- `margin` adds space outside an element.
- `border` adds a border around an element.
- `border-radius` rounds the corners.
- `text-align` aligns text to the left, center, or right.
- `display: flex` lets us arrange items more easily.
- `justify-content` controls horizontal alignment when we use flex.
- `align-items` controls vertical alignment when we use flex.
- `:hover` lets us change the style when we move the mouse over an element.

Attention Please!

The green lines in the CSS block are called **comments**. A comment is a note that does not get executed when the code runs. Its main purpose is to document the code and make it easier for teammates to understand.

Different languages write comments in different ways:

- HTML comment: `<!-- This is a comment -->`
- CSS comment: `/* This is a comment */`
- JavaScript comment: `// This is a comment`

Adding CSS to Our Mini Activity

Now we can return to the same HTML activity and add CSS to it. The HTML still creates the parts, but the CSS makes the activity card look more organized and polished.

HTML with Class Names

```
<div class="activity-card">
  <h2 class="activity-title">Quick Check</h2>
  <p class="activity-question">Which language creates
    the structure?</p>

  <input class="answer-input" type="radio" id="html"
    name="language" value="html">
  <label class="answer-label" for=
    "html">HTML</label><br>

  <input class="answer-input" type="radio" id="css"
    name="language" value="css">
  <label class="answer-label" for="css">CSS</label><br>

  <input class="answer-input" type="radio" id="js" name=
    "language" value="js">
  <label class="answer-label" for=
    "js">JavaScript</label><br>

  <button class="custom-button">Submit</button>
</div>
```

CSS Style Block

```
<style>
.activity-card {
  background-color: #ffefe9;
  border: 2px solid #ff4500;
  border-radius: 12px;
  padding: 20px;
  margin: 20px auto;
  width: 300px;
  text-align: center;
  color: #2d1559;
}

.activity-title {
  color: #ff4500;
  margin-bottom: 10px;
}

.activity-question {
  color: #2d1559;
  font-size: 16px;
}

.answer-input {
  margin: 6px;
}

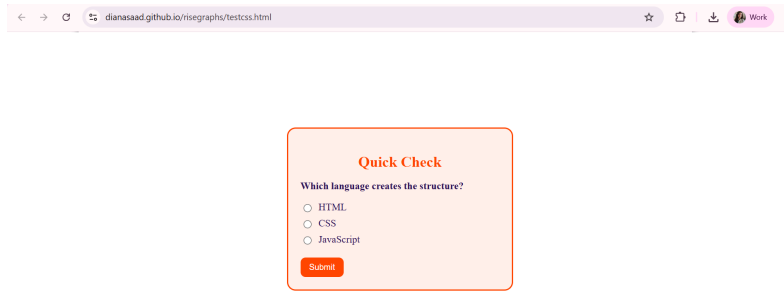
.answer-label {
  color: #2d1559;
}

.custom-button {
  background-color: #ff4500;
  color: white;
  border: none;
  border-radius: 8px;
  padding: 8px 14px;
  margin-top: 12px;
}

/* This is the same class, but this rule applies when we
   hover over the button. */
.custom-button:hover {
  background-color: #2d1559;
}
</style>
```

In this version, we are still using the same HTML structure. The difference is that CSS now gives the card a background color, border, rounded corners, spacing, centered text, and a styled button.

Styled Browser Output



Click the image to open the live styled activity.

Two Ways to Add CSS

When we add CSS, we can write it in more than one way. For now, we will focus on two simple options: **inline CSS** and a **style block**.

- Inline CSS is written directly inside the HTML tag.
- A style block keeps the CSS in one place, which is usually cleaner when we are building a full activity.

Inline CSS

Inline CSS uses the `style` attribute inside the HTML tag.

```
<button class="custom-button" style="background-color:
    #ff4500; color: white;">
  Submit
</button>
```

This works for quick changes, but it can become harder to manage when we have many elements.

Style Block CSS

A style block keeps the CSS separate from the HTML structure.

```
<style>
.custom-button {
  background-color: #ff4500;
  color: white;
}
</style>

<button class="custom-button">Submit</button>
```

This is cleaner because the HTML creates the button, and the CSS controls how the button looks.